Porting Tsigas–Zhang in Mojo

Riassunto del porting, dei test e dei tre benchmark di confronto con Vyukov. Analisi della campagna del 16 ottobre 2026, con 50 ripetizioni per configurazione. Documento preparato sulla struttura del riassunto «Porting RigtorpQueue in Mojo».

1. Implementazione e operazioni della coda

Ho studiato e implementato in Mojo una coda MPMC limitata basata sul protocollo Tsigas–Zhang. MPMC significa che più produttori e più consumatori possono condividere la stessa coda. Il modulo è separato dal Communicator e non è ancora integrato come backend del runtime completo di MoStream.

La coda usa un anello di celle atomiche. Le operazioni cercano la posizione effettiva da leggere o scrivere e possono aiutare ad avanzare le guide head e tail. La pubblicazione e la rimozione del valore avvengono tramite compare-and-swap (CAS) sulla cella. Tutti gli accessi condivisi del porting usano ordinamento sequenzialmente consistente.

Operazione	Comportamento
push(item)	Ripete il tentativo di inserimento con attesa attiva finché riesce; può produrre Error per valori non rappresentabili.
pop()	Attende attivamente e restituisce un valore.
try_push(item)	None al successo; restituisce il valore rifiutato in un Optional se la coda è piena. Può produrre Error.
try_pop()	Restituisce un Optional con il valore, oppure None se la coda è vuota.
size() / empty()	Eseguono una scansione; il conteggio è esatto solo senza accessi concorrenti.

A differenza della coda Rigtorp generica, questo porting contiene solo UInt64. I valori 0 e 1 sono riservati internamente ai marcatori di vuoto: il valore applicativo x viene codificato come x+2. Sono quindi accettati i valori da 0 a 2^64−3 inclusi; i due valori superiori vengono rifiutati prima di modificare la coda.

Il costruttore riceve la lunghezza fisica L dell’anello, che deve essere una potenza di due almeno pari a 4. La capacità utile è L−2: un anello di 1024 celle contiene al massimo 1022 elementi. Non avvengono allocazioni per elemento durante push e pop.

2. Test funzionali e limite di correttezza

Nel test sequenziale ho verificato anelli da 4, 8, 16 e 64 celle, con 200 cicli di riempimento e svuotamento per ciascuno. Sono controllati ordine FIFO, pieno e vuoto, valori ripetuti, zero e uno, massimo valore rappresentabile, spostamento di proprietà, distruzione di una coda non vuota e rifiuto dei parametri non validi.

Sei scenari concorrenti usano quattro produttori e quattro consumatori, con operazioni bloccanti e try_*, anelli da 64 e 16384 celle e partenze anche vicine al confine fisico. Un registro atomico verifica che ogni identificativo sia ricevuto esattamente una volta. Si aggiunge una regressione SPSC con un produttore e un consumatore, che verifica l’ordine esatto di 2000 valori. In totale i sette scenari trasferiscono 18.032 valori.

La validazione registrata il 16 ottobre 2026 con Mojo 1.1.0 e ottimizzazione -O3 risulta superata. Non ho rieseguito i test per preparare questa relazione: riporto la validazione conservata nel progetto. Durante lo sviluppo è stata inoltre rafforzata la scansione di try_pop: si acquisisce tail prima di leggere la cella e si ricomincia se cambia head oppure tail, per evitare di saltare una pubblicazione concorrente.

Il superamento dei test verifica i casi eseguiti, ma non dimostra correttezza sotto sospensioni arbitrarie. Il protocollo conserva un limite ABA: dopo abbastanza giri dell’anello, indici e marcatori possono tornare uguali a quelli osservati da un’operazione rimasta sospesa. L’ordinamento sequenzialmente consistente non elimina questo limite. Il backend rimane quindi sperimentale.

3. Impostazione dei benchmark

Ho analizzato PipelineGrain, FarmThroughput e ImageTiles: rispettivamente 2000, 2000 e 1550 misure, per un totale di 5550 esecuzioni. Ogni configurazione ha 50 ripetizioni. I controlli di correttezza della campagna risultano superati. La mediana riassume le prove senza dare lo stesso peso della media ai valori estremi; non elimina però la variabilità.

Per limitare il problema ABA, ogni misura usa code nuove con un anello sufficientemente grande da non compiere un giro completo durante la prova. Pipeline e farm usano 131.072 celle fisiche; ImageTiles ne usa 65.536 per coda. La lunghezza fisica è identica per i backend, ma i posti utili sono L−2 per Tsigas e L per Vyukov. Sono misure a lotti su anelli grandi: non rappresentano il riuso continuo di una piccola coda piena.

Il kernel sintetico applica a ogni iterazione una maschera a 63 bit, identica per i due backend, per evitare valori non rappresentabili. Questa modifica e le capacità diverse impediscono un confronto numerico diretto con la precedente campagna Rigtorp da 1024 posti. Il confronto valido qui è Tsigas contro Vyukov nelle medesime configurazioni della nuova campagna.

Per pipeline e farm sono disponibili intervalli bootstrap al 95% del rapporto di throughput Tsigas/Vyukov, con 20.000 ricampionamenti. Valori superiori a 1 favoriscono Tsigas; inferiori a 1 favoriscono Vyukov. Gli intervalli sono individuali, senza correzione per confronti multipli: l’analisi è esplorativa. ImageTiles presenta statistiche descrittive, senza intervalli di confidenza del rapporto.

4. PipelineGrain

PipelineGrain collega N stadi sequenziali mediante N−1 code. Ogni input attraversa tutti gli stadi; il lavoro totale rimane fisso e viene suddiviso tra gli stadi. Con N=1 non viene usata alcuna coda: è una baseline, non un confronto tra algoritmi di coda.

I tempi target del lavoro totale per input sono T=0, 1, 2 e 5 µs. Il riferimento ideale è il costo misurato con N=1 diviso N. Il costo in ns/input deriva dal tempo totale del lotto diviso il numero di input: descrive l'inverso del throughput, non la latenza di attraversamento del singolo input.

Con T=0 non c'è calcolo utile da distribuire. Tsigas è più veloce con due stadi (circa 197 contro 253 ns/input), mentre Vyukov prevale con 4, 8 e 16 stadi. Entrambe le pipeline sono molto più costose della baseline senza code: un vantaggio relativo tra code non rende conveniente la frammentazione di un lavoro nullo.

T=0 µs; N	Tsigas ns/input	Vyukov ns/input	Throughput T/V
2	197	253	1,284
4	483	408	0,844
8	604	522	0,864
16	713	612	0,859

Con T=1 µs Tsigas ha tempi inferiori con 2, 4 e 8 stadi; a 16 prevale Vyukov. Il miglior punto Tsigas è N=2, circa 658 ns/input e speedup 1,51×. Aumentare ulteriormente gli stadi non porta un miglioramento sistematico.

T=1 µs; N	Tsigas ns/input	Vyukov ns/input	Throughput T/V
2	658	835	1,269
4	684	831	1,215
8	730	780	1,068
16	702	668	0,952

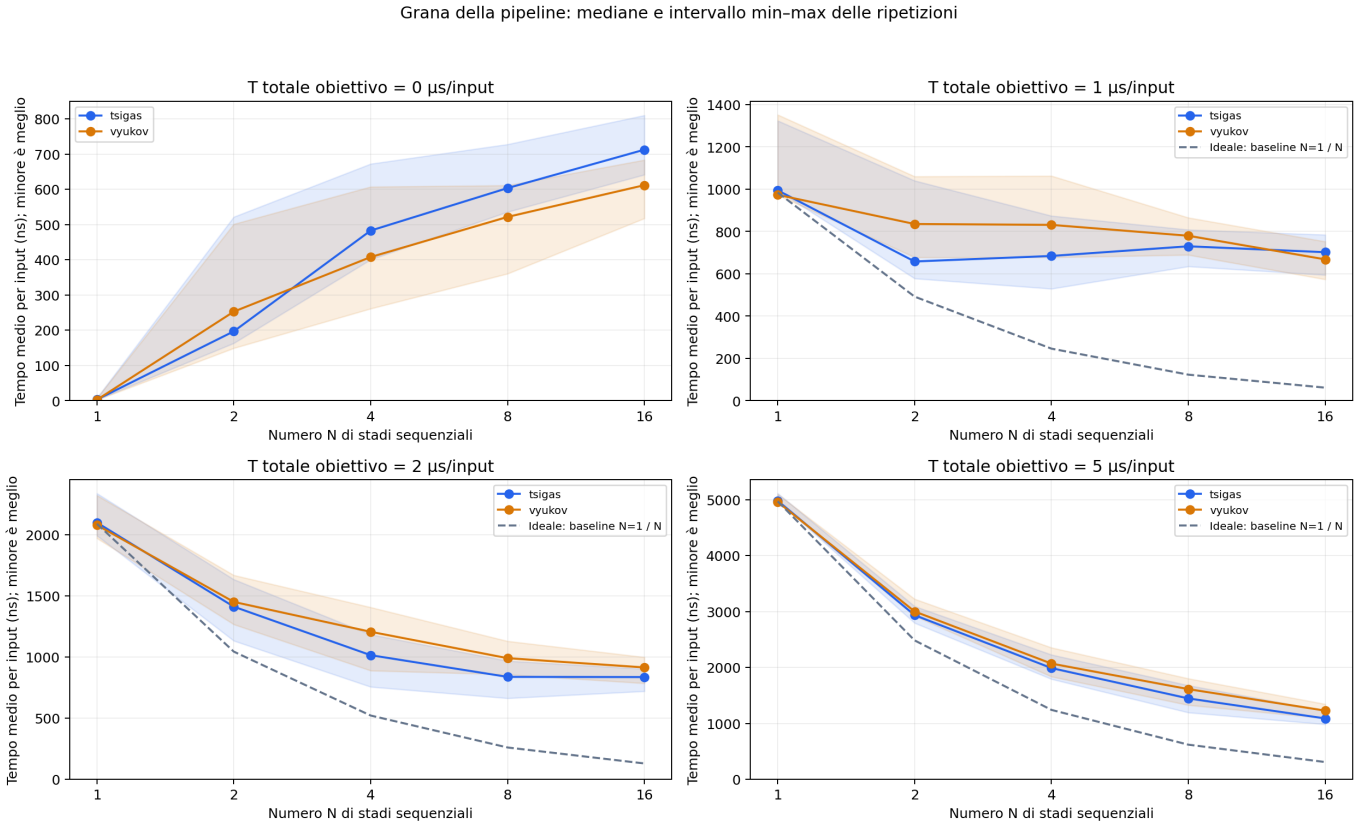
Con T=2 µs Tsigas ha mediane inferiori per tutti gli N>1, ma a N=2 l'intervallo del rapporto comprende la parità. Con 4, 8 e 16 stadi gli intervalli individuali sostengono un vantaggio di throughput superiore al 5%. A 16 stadi si misurano circa 836 contro 915 ns/input: il throughput Tsigas è maggiore di circa il 9,4%, con un tempo inferiore di circa l'8,6%.

T=2 µs; N	Tsigas ns/input	Vyukov ns/input	Throughput T/V
2	1411	1450	1,028
4	1015	1206	1,188
8	838	990	1,183
16	836	915	1,094

Con T=5 µs il parallelismo è più utile. A 16 stadi Tsigas misura circa 1090 ns/input contro 1229 ns di Vyukov: throughput superiore di circa il 12,8%. Gli speedup sulle rispettive baseline N=1 sono 4,57× e 4,03×. Il riferimento ideale a 16 stadi rimane lontano: l’efficienza Tsigas è circa il 28,6%.

T=5 µs; N	Tsigas ns/input	Vyukov ns/input	Throughput T/V
2	2936	3002	1,022
4	1990	2070	1,040
8	1448	1614	1,115
16	1090	1229	1,128

pipeline grain



PipelineGrain: costo per input al variare degli stadi. Blu = Tsigas, arancione = Vyukov. Le fasce min-max descrivono l'escursione osservata, non intervalli di confidenza. Il riferimento ideale usa la baseline N=1 divisa per N.

5. FarmThroughput

La farm usa una sorgente, una coda condivisa e N consumatori: sorgente → coda lavori → N consumatori. A differenza della pipeline, ogni input viene elaborato da un solo consumatore, che esegue tutto il lavoro T. Il throughput conta gli input completati al secondo; N indica solo i consumatori e il totale dei task è N+1.

Con T=0 Tsigas passa da circa 4,83 milioni di input/s con un consumatore a 0,25 milioni con 16. Vyukov passa da 3,68 a 1,38 milioni. Tsigas prevale con un consumatore, ma già da due consumatori Vyukov è più veloce. La condivisione della coda penalizza fortemente questo porting quando il lavoro è minimo.

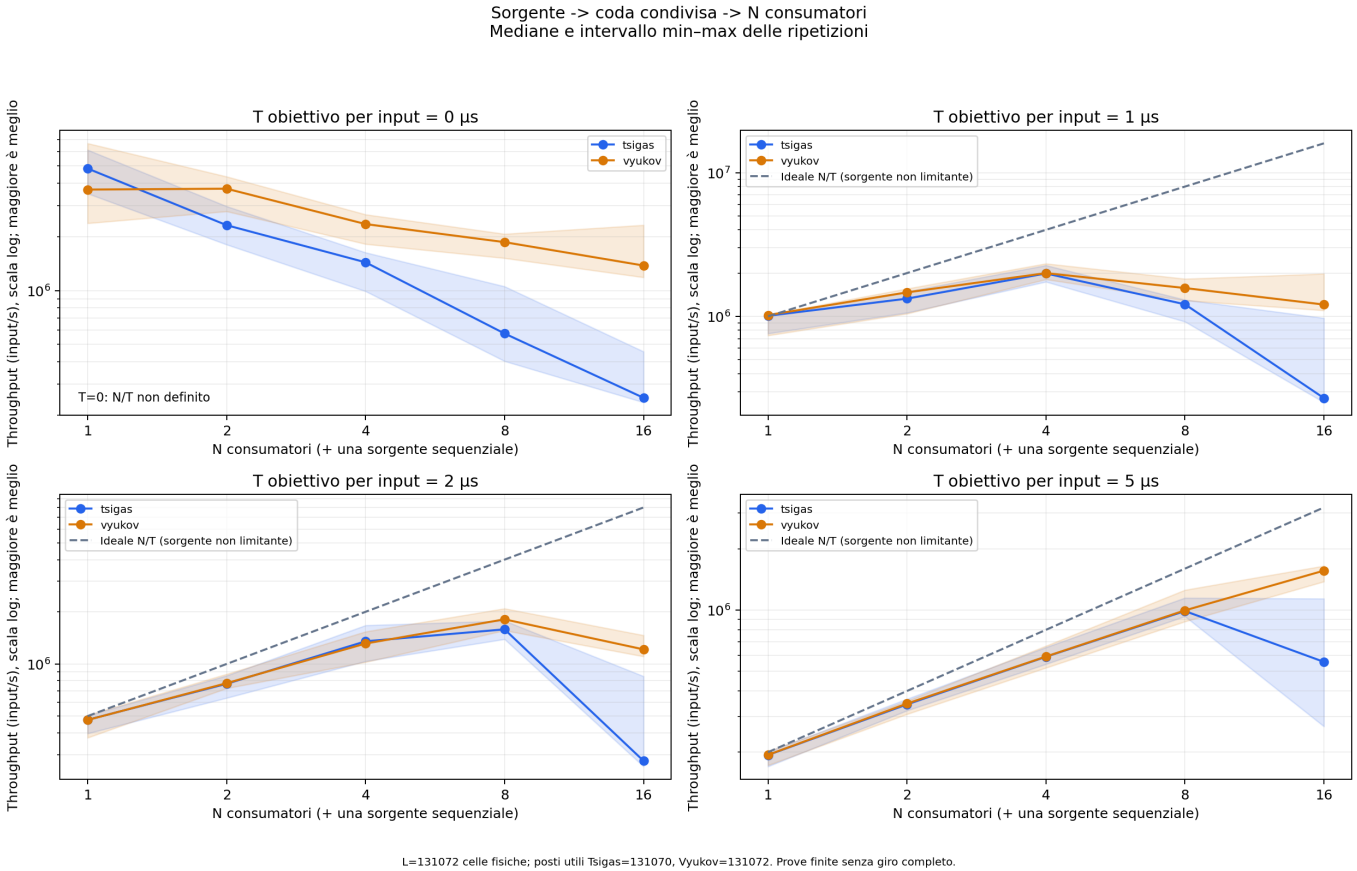
Con T=1 µs entrambi raggiungono il miglior throughput a quattro consumatori, circa 1,99 milioni di input/s per Tsigas e 2,00 milioni per Vyukov. A 16 consumatori Tsigas scende a 0,27 milioni, contro 1,21 milioni di Vyukov.

Con T=2 µs il miglior punto è N=8 per entrambi: 1,58 milioni di input/s per Tsigas e 1,81 milioni per Vyukov. A N=4 Tsigas ha un piccolo vantaggio descrittivo, ma l’intervallo non sostiene un margine superiore al 5%. Passare a 16 consumatori peggiora entrambi, soprattutto Tsigas.

Con T=5 µs le due code restano vicine fino a N=8. Tsigas raggiunge il proprio massimo a otto consumatori, circa 0,99 milioni di input/s, poi scende a 0,56 milioni con 16. Vyukov continua a crescere fino a 1,56 milioni. Il rapporto Tsigas/Vyukov a N=16 è circa 0,356: Tsigas completa circa il 64,4% di input in meno per secondo, oppure Vyukov ha un throughput circa 2,81 volte maggiore.

T µs	Coda	N migliore	Max milioni input/s	A N=16
0	tsigas	1	4,827	0,251
0	vyukov	2	3,721	1,381
1	tsigas	4	1,988	0,270
1	vyukov	4	2,000	1,208
2	tsigas	8	1,584	0,276
2	vyukov	8	1,810	1,214
5	tsigas	8	0,990	0,556
5	vyukov	16	1,564	1,564

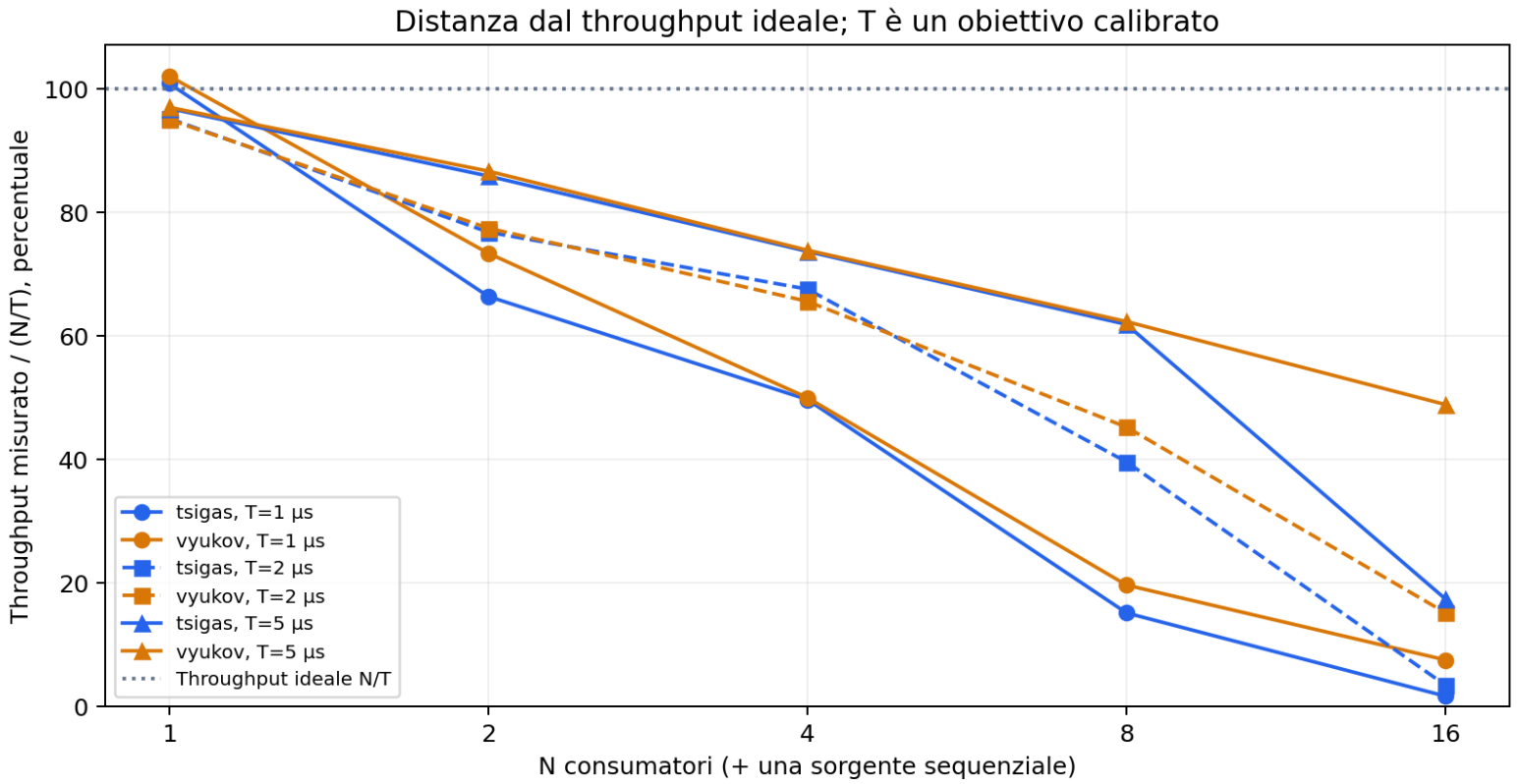
farm throughput



FarmThroughput: più alto è il throughput, meglio è. I punti derivano dai tempi mediani; le fasce sono min-max. La caduta di Tsigas con molti consumatori riguarda queste configurazioni e non identifica da sola il meccanismo responsabile.

L’efficienza rispetto al target si calcola come throughput misurato / (N/T), moltiplicato per 100; T deve essere espresso in secondi. Non è definita per T=0. Con T=5 µs e N=16 Tsigas è circa al 17,4% dell’ideale, Vyukov al 48,9%. A N=8 sono invece circa al 61,9% e al 62,3%. Un valore appena superiore al 100% può dipendere dal target calibrato, che non coincide necessariamente con il costo effettivo misurato.

farm efficiency



L=131072 celle fisiche; posti utili Tsigas=131070, Vyukov=131072. Prove finite senza giro completo.

Efficienza della farm rispetto a N/T. T è un obiettivo calibrato. Una percentuale inferiore non implica necessariamente meno throughput assoluto; qui occorre leggere anche il grafico precedente.

6. ImageTiles

ImageTiles elabora immagini RGB sintetiche 512×512, generate in memoria, convertendole in scala di grigi. Il percorso è: produttore → coda lavori → N consumatori → coda completamenti → raccoglitore. La formula è gray=(77R+150G+29B)>>8; il grigio viene scritto nei tre canali. Seriale e parallelo usano lo stesso kernel SIMD.

Ogni immagine viene divisa in strisce orizzontali da 1, 16 o 128 righe, corrispondenti a 512, 32 o 4 blocchi per immagine. Ogni prova elabora 32 immagini. Le code trasportano identificativi UInt64, non i pixel. I consumatori scrivono regioni disgiunte del buffer finale; il raccoglitore certifica il completamento quando riceve tutti i blocchi, senza copia finale.

Le configurazioni usano N=1, 2, 4, 8 e 16 consumatori, più produttore e raccoglitore, per 50 ripetizioni. Il throughput è 32 diviso il tempo totale mediano in secondi. La baseline seriale raggiunge circa 6925 immagini/s, con latenza mediana di servizio di circa 0,144 ms. Allocazione dei buffer e verifica byte per byte sono fuori dal timer; comunicazione e raccolta sono misurate.

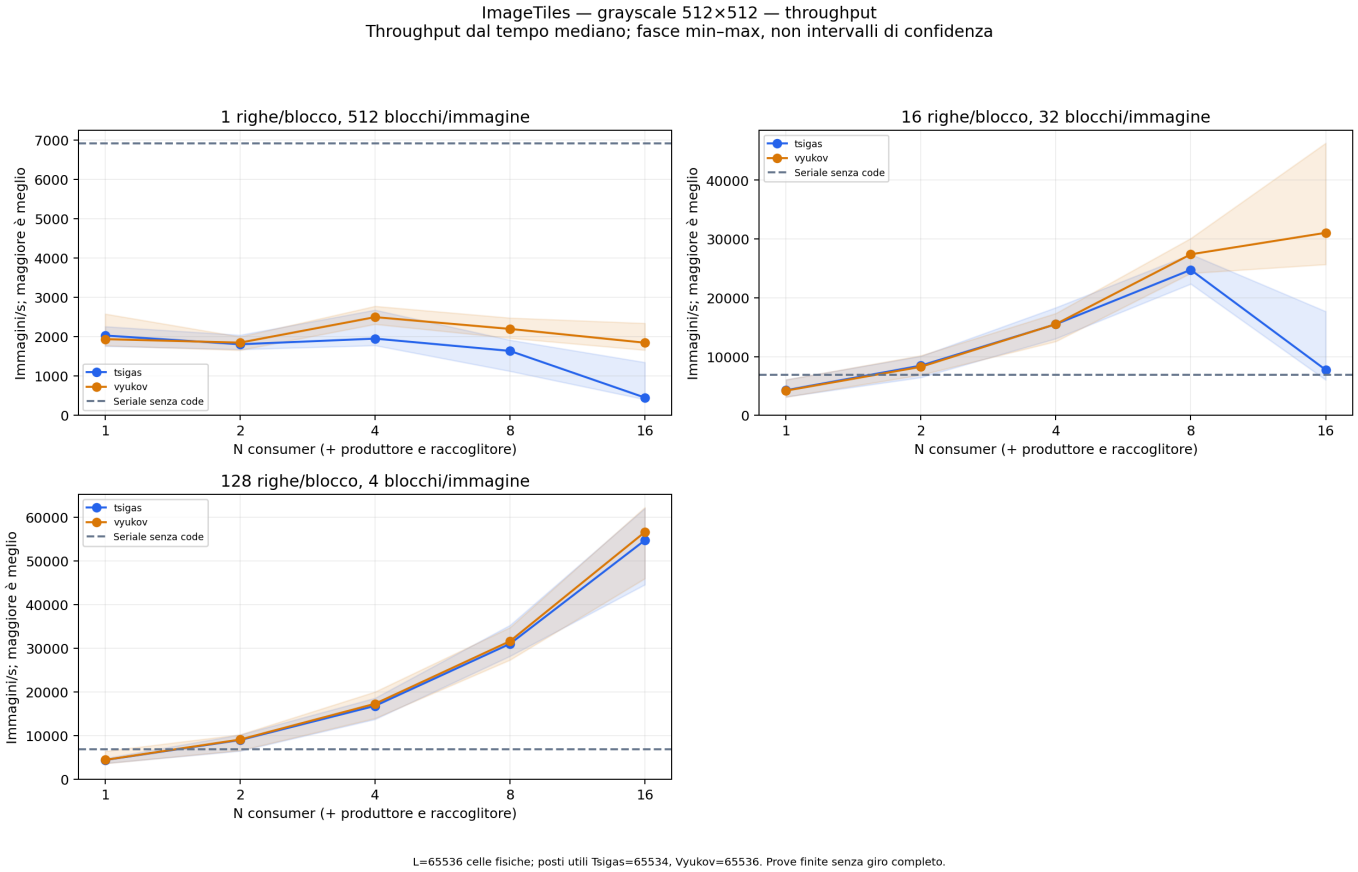
Con blocchi da una riga nessuna configurazione supera il seriale. Tsigas raggiunge circa 2031 immagini/s con un consumatore; a 16 scende a 451 immagini/s, contro 1847 di Vyukov. Il miglior punto Vyukov è N=4, circa 2499 immagini/s. La comunicazione frequente rende questa suddivisione poco conveniente nel benchmark osservato.

Con blocchi da 16 righe Tsigas cresce fino a otto consumatori, arrivando a circa 24.753 immagini/s e speedup 3,57× sul seriale. A 16 consumatori scende a 7733 immagini/s, mentre Vyukov raggiunge circa 31.052 immagini/s e speedup 4,48×. Quindi il vantaggio Vyukov a N=16 è di circa quattro volte nel throughput.

Con blocchi da 128 righe le prestazioni delle due code sono vicine e il throughput cresce fino a N=16. Tsigas raggiunge circa 54.715 immagini/s, con speedup 7,90×; Vyukov circa 56.590 immagini/s, con speedup 8,17×. La differenza descrittiva è di circa il 3,3% a favore di Vyukov; senza intervalli dedicati non la tratto come una superiorità statisticamente dimostrata. Pur essendoci solo quattro blocchi per immagine, più immagini possono essere elaborate contemporaneamente.

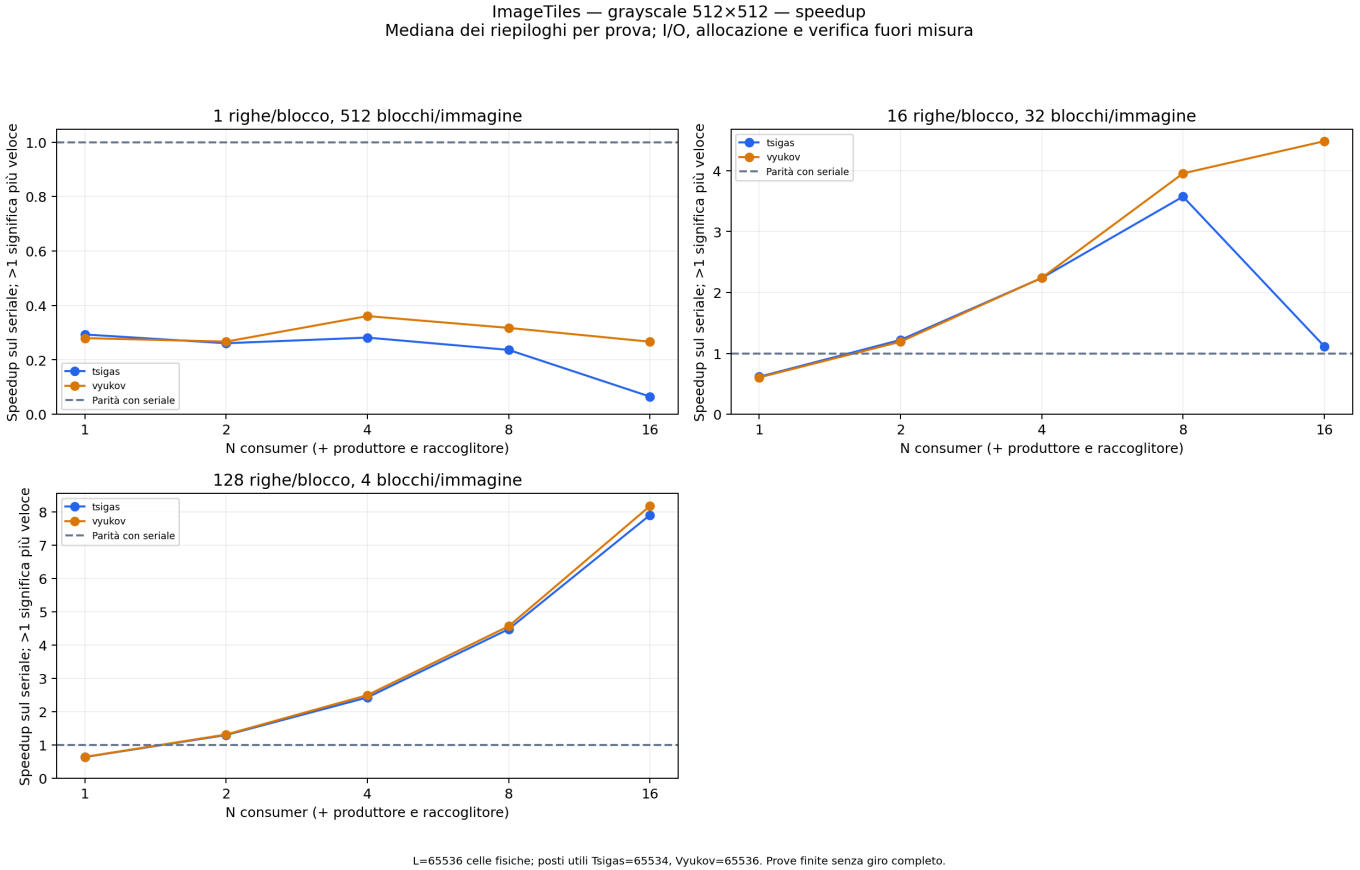
Righe	N	Tsigas img/s	Vyukov img/s	Speedup T	Speedup V
1	1	2031	1939	0,29	0,28
1	4	1952	2499	0,28	0,36
1	8	1639	2199	0,24	0,32
1	16	451	1847	0,07	0,27
16	1	4266	4193	0,62	0,61
16	4	15499	15515	2,24	2,24
16	8	24753	27391	3,57	3,96
16	16	7733	31052	1,12	4,48
128	1	4442	4472	0,64	0,65
128	4	16825	17310	2,43	2,50
128	8	31016	31625	4,48	4,57
128	16	54715	56590	7,90	8,17

throughput 512x512



ImageTiles: throughput per dimensione dei blocchi. Le scale verticali possono differire tra pannelli. Con blocchi da 128 righe i due backend sono vicini; con 1 e 16 righe Tsigas peggiora a N=16.

speedup 512x512



ImageTiles: speedup sul seriale. Un valore superiore a 1 indica un tempo totale del lotto inferiore al seriale. Con una riga per blocco tutte le configurazioni restano sotto 1.

7. Interpretazione della latenza di ImageTiles

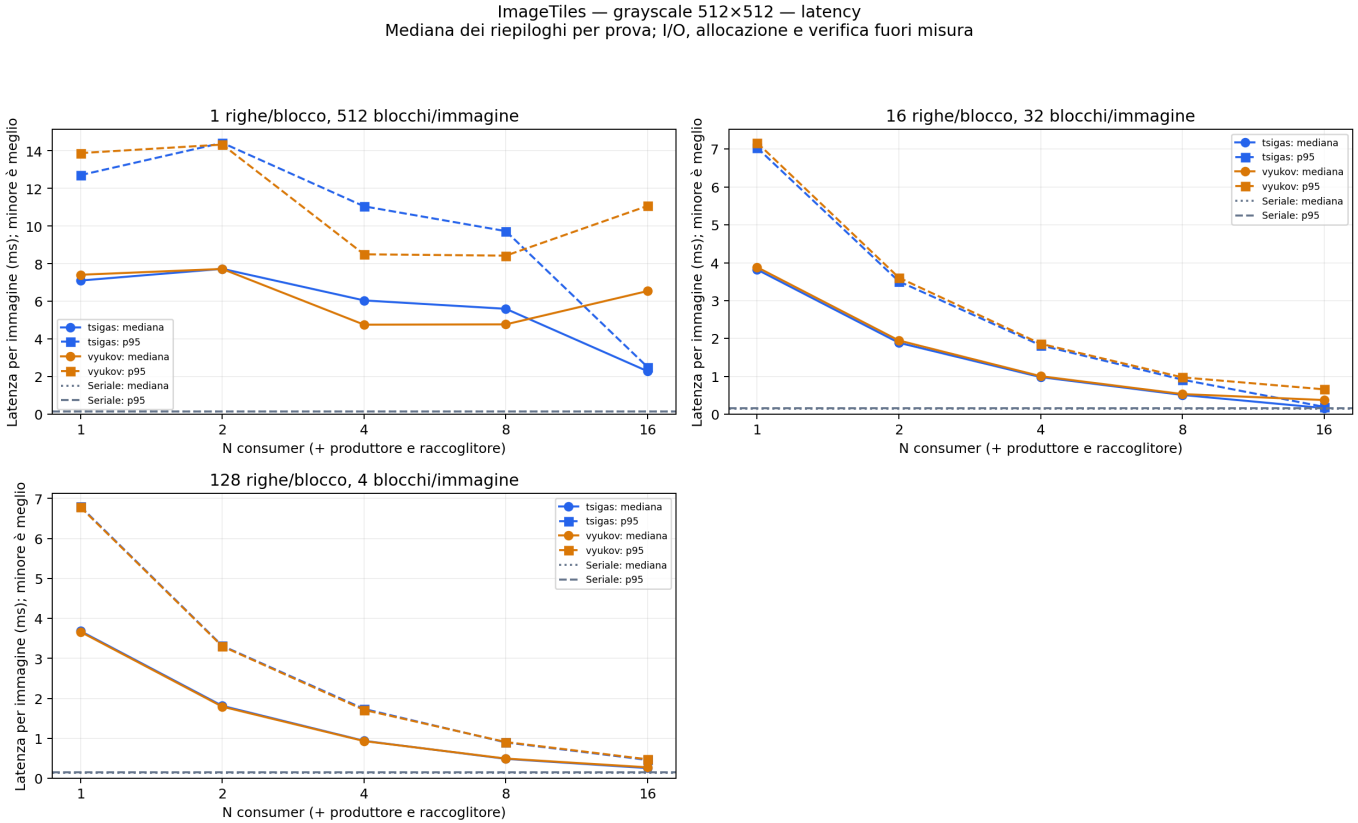
La latenza parallela va dall’inizio dell’invio del primo blocco di un’immagine alla ricezione dell’ultimo completamento. Include backpressure e attese nelle code, ma esclude l’attesa precedente all’inizio dell’invio. La latenza seriale è invece il tempo di servizio del kernel per immagine. Non sono tempi di risposta misurati rispetto allo stesso flusso esterno di arrivi.

Le linee continue del grafico mostrano la mediana; quelle tratteggiate il p95, entro cui termina circa il 95% delle immagini della singola prova. Ogni punto è poi la mediana dei rispettivi riepiloghi delle 50 prove. La distanza tra mediana e p95 non è un intervallo di confidenza.

Il caso più istruttivo è Tsigas con blocchi da 16 righe: da N=8 a N=16 la latenza mediana scende da circa 0,511 a 0,164 ms, ma il throughput cala da 24.753 a 7733 immagini/s. Anche con blocchi da una riga la mediana scende da circa 5,61 a 2,29 ms, mentre il throughput passa da 1639 a 451 immagini/s. Una latenza inferiore, quindi, non basta a dichiarare migliore la configurazione.

Questi dati sono compatibili con un’ammissione più lenta delle immagini: se il produttore inizia a inviarle più tardi, una parte del ritardo resta prima del timestamp iniziale e non viene contata nella latenza. Il grafico da solo non prova questa causa; servirebbero misure dei tempi di emissione, della coda e degli intervalli tra gli arrivi. Latenza e throughput devono essere letti insieme.

latency 512x512



ImageTiles: latenza dall'emissione al completamento, non dal medesimo arrivo esterno. Il calo della latenza Tsigas a N=16 nei pannelli con blocchi piccoli accompagna un calo del throughput: non è una dimostrazione di miglioramento complessivo.

8. Conclusioni e collegamento al runtime cooperativo

Dai risultati non emerge una superiorità generale di Tsigas. Nella pipeline, dove ciascuna coda collega due stadi e ha un produttore e un consumatore, il porting mostra vantaggi in diverse configurazioni con calcolo utile, soprattutto T=2 e 5 µs. Nella farm e in ImageTiles con blocchi piccoli, invece, la coda condivisa tra molti consumatori mostra un forte peggioramento ad alto parallelismo.

La granularità è decisiva: con blocchi da 128 righe ImageTiles ammortizza la comunicazione e i backend diventano vicini. Nei sintetici, aumentare gli stadi o i consumatori non è sempre utile. La topologia degli accessi e il rapporto tra calcolo e comunicazione contano quanto la scelta della coda.

Per un runtime cooperativo questi risultati suggeriscono di studiare code meno contese, un numero contenuto di worker e il raggruppamento di task troppo piccoli in unità di lavoro più grandi. Sono ipotesi progettuali, non prestazioni già misurate: gli harness impiegano attesa attiva e non il runtime cooperativo completo. Push e pop bloccanti possono occupare un worker mentre altre attività dovrebbero essere eseguite.

Prima di un’integrazione serve inoltre affrontare il vincolo di correttezza sotto sospensioni: le prove finite su anelli grandi evitano una condizione problematica, ma non dimostrano che il backend sia adatto a un’esecuzione continua con riuso dell’anello. Il passo successivo è definire il contratto richiesto dal runtime e poi confrontare le implementazioni al suo interno, con un carico e una politica di scheduling comuni.

9. Fonti e tracciabilità

Fonte dei valori: Benchmarks/BenchMark Tsigas vs Vyukov/Campagna_50_ripetizioni/risultati/20261016-campagna-50/. Per ogni benchmark sono stati consultati summary.csv, raw.csv e metadata.json; per i sintetici anche confronto_50.md. VERIFICA.json registra i conteggi e gli hash della campagna. I grafici incorporati sono quelli originali Tsigas/Vyukov di questa cartella, non grafici Rigtorp rinominati.

Fonti del porting e dei test: MoStream/MPMC_queue_tsigas_zhang.mojo; docs/Tsigas_Zhang_Mojo.md; docs/Tsigas_Zhang_validazione.json; Tests/test_tsigas_zhang.mojo; Tests/test_tsigas_zhang_concurrent.mojo; Tests/run_tsigas_zhang_tests.py. Il documento su Rigtorp è stato usato come riferimento di struttura, non come fonte dei risultati Tsigas.